

程序设计实习大作业报告

一、程序功能介绍

（一）、核心交易功能：实现商品发现与交易闭环

1. 商品浏览与搜索：

平台提供多样化的商品发现途径。用户可以通过预设的分类浏览结构（如按商品类别、品牌等划分）系统地探索商品库。关键词搜索功能允许用户输入具体的商品名称、特性或描述词汇，快速定位相关商品。在浏览和搜索结果页，强大的筛选功能可供用户根据多种条件（例如价格范围、商品状态、地理位置等可用选项）进一步缩小范围，精准定位符合个人需求的商品。这三者结合，确保了用户高效找到目标商品。

2. 商品发布：

卖家享有便捷的商品上架流程。该流程设计简洁直观，引导卖家逐步完成信息填写。

发布功能支持多图上传，卖家可以添加多张图片以多角度、全方位展示商品细节。

卖家能够设置详细的商品描述，包括商品规格、使用情况、瑕疵说明等关键信息，确保信息透明。核心功能包含价格设置，卖家可为商品设定明确的出售价格。这是促成交易的基础信息。

3. 收藏功能：

用户可将浏览过程中感兴趣的、但可能暂不购买的商品添加至个人收藏夹。此功能的主要目的是方便用户后续查看这些商品，无需重复搜索，便于集中管理和比较。

4. 评价系统：

在交易流程顺利完成后，系统会引导买卖双方互相进行评价。用户基于实际交易体验（如商品符合描述程度、沟通顺畅度、交易效率等）给出评价。这些评价公开可见，共同构成了平台的信用体系，为其他用户提供交易参考，促进诚信交易环境的建立。

（二）、社交与账户管理：连接用户，掌控信息

1. 个人账户：

用户拥有独立的个人账户中心，用于管理个人信息（如头像、昵称、联系方式等基础资料）。账户中心为用户查看交易记录的核心入口，清晰展示其作为买家或卖家的所有历史订单和当前交易状态。对于卖家用户，账户中心提供发布商品管理功能，可以查看、上架、下架自己发布的所有商品信息。

2. 私信沟通：

平台内置点对点的私信沟通渠道，允许买家与卖家在交易前后直接进行交流。该功能支持用户发送文字消息进行沟通。同时，也支持用户发送图片消息（例如发送商品细节图、凭证截图等），使沟通更直观高效，解答疑问，促成交易。

3. 编辑功能：

平台赋予用户灵活修改信息的权限。卖家可以随时编辑已发布的商品信息（包括图片、描述、价格等），以更新商品状态或修正信息。用户也可以随时编辑个人资料信息，确保个人展示信息的准确性和时效性。

（三）、其他辅助功能：提升使用便捷与安全性

1. 收藏夹管理：

用户可以对添加到收藏夹的商品进行整理和管理操作。这包括查看收藏列表、移除不再感兴趣的商品等基础管理功能，帮助用户保持收藏夹的有效性和条理性。此外，平台发布的系统通知（如公告、规则更新提示等）也会在此显示，确保用户及时知晓平台动态。

2. 安全认证：

平台提供基础的账户登录与注册功能，是用户访问和使用服务的前提。配套的安全设置选项（如修改密码、绑定手机/邮箱验证等）是核心组成部分，旨在帮助用户保护账户安全，防止未授权访问。

二、项目各模块与类设计细节

（一）、chatmessage 类

1. 构造函数 ChatMessage::ChatMessage()

功能：初始化聊天消息对象，记录消息的基本信息和发送时间。

内部操作：

- 将传入的发送者、接收者和内容赋值给对象成员
- 自动将 `sendTime` 设置为系统当前时间

2. `toJson()` 成员函数

功能：将消息对象转换为 JSON 格式，用于序列化传输或存储。

关键处理：时间字段使用 Qt::ISODate 标准格式（如 2025-06-30T15:30:00）

3. `fromJson()` 静态成员函数

功能：从 JSON 对象解析并重建消息对象，实现反序列化。

处理流程：

- （1）. 提取 `sender`、`receiver` 和 `content` 字段
- （2）. 将 ISO 格式的 `sendTime` 字符串转换为 QDateTime 对象

4. `saveMessages()` 静态成员函数

功能：将多条消息保存到本地文件（JSON 格式）。

操作流程：

- （1）. 将每条消息转为 JSON 对象并加入数组
- （2）. 创建 JSON 文档并写入文件
- （3）. 文件以只写模式打开

5. `loadMessages()` 静态成员函数

功能：从文件加载历史消息列表。

###整体设计要点

1. 数据持久化

- 使用 JSON 格式实现消息的本地存储与读取

2. 时间标准化

- 统一采用 ISO8601 时间格式确保跨平台兼容性

3. 错误静默处理

- 文件操作失败时返回错误标识（非中断式处理）

4. 静态方法设计

- 消息保存/加载功能不依赖对象实例，直接通过类调用

（二）Chatroom 函数

1. 构造函数 `ChatRoom::ChatRoom()`

功能：初始化聊天室窗口，建立两个用户间的聊天环境。

关键操作：

- （1）. 调用 `setupUI()` 初始化界面
- （2）. 生成历史消息和未读状态文件路径
- （3）. 按顺序加载未读状态和历史消息
- （4）. 输出调试信息（用户标识、文件路径等）

2. `setupUI()` 私有成员函数

功能：

构建聊天室界面布局和基本交互逻辑。

3. `addMessage()` 公有成员函数

功能：添加新消息到聊天室并更新相关状态。

处理流程：

- （1）. 创建 `ChatMessage` 对象并存入历史记录
- （2）. 在界面显示消息

(3) . 若非当前用户发送:

- 增加接收方未读计数
- 更新窗口标题
- 保存未读状态
- 发射 `unreadCountChanged` 信号

(4) 保存历史

4. `updateTitle()` 私有成员函数

功能: 动态更新窗口标题栏显示内容。

标题格式: 聊天室 - [用户 1] 与 [用户 2] (X 条未读)

未读提示: 仅当当前用户有未读消息时显示计数

5. `sendMessage()` 槽函数

功能: 处理用户发送消息操作。

6. `displayMessage()` 私有成员函数

功能: 在消息显示区渲染单条消息。

7. 历史消息管理函数

`loadHistory()`

- 从 JSON 文件加载历史记录
- 清空显示区后重新渲染所有消息
- 不修改未读计数

`saveHistory()`

- 将消息列表保存为 JSON 文件
- 失败时显示警告对话框

`getHistoryFilePath()`

- 返回格式化的历史文件路径
- 路径规则: `chat_history/[用户 1]_[用户 2].json`
- 自动处理用户名顺序无关性

8. 未读状态管理函数

`loadUnreadState()`

- 从 JSON 文件加载未读计数
- 默认值: 0 (文件不存在时)
- 加载后更新窗口标题

`saveUnreadState()`

- 保存当前未读计数到 JSON 文件
- 文件路径: `chat_unread_state/[用户 1]_[用户 2]_unread.json`

`getUnreadStateFilePath()`

- 生成未读状态文件路径
- 处理用户名顺序无关性

9. 未读标记操作函数

`markAsRead(User*)`

- 将指定用户的未读计数归零
- 更新标题并保存状态
- 发射 `unreadCountChanged` 信号

`markAsUnread(User*)`

- 递增指定用户的未读计数

- 更新标题并保存状态
- 发射 `unreadCountChanged` 信号

`getUnreadCount(User*)`

- 查询指定用户的未读消息数 • 返回 `unreadCount1` 或 `unreadCount2`

10. 辅助函数

`hasUser(User*)`

- 检查用户是否参与当前聊天室
- 通过用户 ID 比对判断

`getCurrentTime()`

- 返回当前时间的格式化字符串 (`yyyy-MM-dd hh:mm:ss`)

###关键设计说明

1. 双用户管理

- 通过 `user1` 和 `user2` 区分聊天双方
- 自动处理用户名排序问题 (文件路径生成)

2. 状态持久化

- 历史消息与未读状态分离存储
- 使用 JSON 格式保证可读性

3. 实时反馈机制

- 通过 `unreadCountChanged` 信号通知外部未读变化
- 窗口标题动态更新未读计数

4. 调试支持

- 关键操作均输出 `QDebug()` 日志

(三)、`editdialog`

1. 构造函数 `EditDialog::EditDialog()`

功能: 初始化用户信息编辑对话框界面。

布局结构: 表单布局(`QFormLayout`)包含所有输入字段, 底部按钮组(`QDialogButtonBox`)包含确定/取消按钮

钮

信号连接: 确定按钮: 触发 `validateInput()` 验证

取消按钮: 直接关闭对话框

2. `validateInput()` 私有槽函数

功能: 验证用户输入数据的有效性。

3. 数据获取函数

`getUsername()`

返回: 去除首尾空格的用户名输入内容

用途: 供外部获取修改后的用户名

`getEmail()`

返回: 去除首尾空格的邮箱输入内容

用途: 供外部获取修改后的邮箱

`getNewPassword()`

返回: 密码框原始内容 (未 `trim` 处理)

###关键设计特点

密码修改可选性, 密码框留空视为不修改密码, 避免强制密码更新, 即时验证反馈, 用 `QMessageBox` 实时显示错误信息, 数据安全处理, 密码输入框始终启用密码隐藏模式 (`EchoMode::Password`), 内存管理, 通过 `a=NULL` 显式清除临时用户指针 (实际可省略, 此处为示范性代码)

（四）、findpost 函数

功能概述：该函数用于在商品帖子集合中执行多条件搜索，返回符合匹配条件的帖子迭代器列表。

搜索功能

1. 标题搜索：检查帖子标题是否包含关键词
2. 标签搜索：检查帖子标签是否包含关键词
3. 描述搜索：检查帖子描述是否包含关键词

搜索特点

- 采用"或"逻辑匹配（标题/标签/描述任一匹配即可）
- 不区分大小写
- 返回匹配帖子的迭代器

（五）、itempost 函数

1. 类功能概述

ItemPost 类用于表示和管理商品帖子信息，提供完整的数据封装和操作方法。

2. 核心功能

2.1 基础信息管理

- 存储商品标题、描述、价格等基本信息
- 记录商品发布者和发布时间
- 管理商品标签集合

2.2 媒体资源管理

- 维护商品图片集合
- 支持设置主图功能
- 提供图片删除功能

2.3 用户互动功能

- 实现收藏功能
- 支持添加/移除收藏
- 提供收藏状态查询

3. 数据持久化

- 完整 JSON 序列化支持
- 包含所有关键字段的转换
- 支持从 JSON 恢复对象

（六）、logindialog 类

1. 核心功能亮点

- 实现完整的用户登录验证流程
- 支持密码隐藏显示(*****)
- 提供即时的登录反馈(成功/失败提示)

2. 代码设计亮点

- 采用 Lambda 表达式处理按钮点击事件
- 直接访问全局用户列表进行验证
- 自动记录登录成功的用户信息
- 使用模态对话框实现登录流程控制

3. 交互设计亮点

- 简洁的表单式布局(FormLayout)
- 即时的错误反馈机制
- 合理的窗口尺寸控制(250x150)

4. 安全特性

- 密码自动掩码保护
- 安全的密码比对方式(`checkPassword`)
- 验证失败不清除输入(方便重新尝试)

5. 集成特性

- 与全局用户系统无缝对接
- 通过 `accept()` 返回登录状态
- 自动更新当前用户指针

(七)、`ratingdialog` 函数

1. 基本功能

- 提供 1-5 分的评分功能（滑块+数字输入框）
- 支持输入文字评价内容
- 实时显示评分对应的星级和评价文字
- 包含提交和取消按钮

2. 交互特点

- 滑块和数字输入框双向同步
- 五种评分级别对应不同星级显示
- 美观的现代化界面设计
- 响应式布局适应不同尺寸

3. 技术亮点

- 使用 QSS 实现精美界面样式
- 信号槽机制实现组件联动
- 模块化的 UI 搭建方式
- 严格的输入验证机制

4. 使用方法

- 构造时传入被评价用户名
- 调用 `exec()` 显示对话框
- 通过 `getRating()` 获取评分(1-5)
- 通过 `getComment()` 获取评价文字

5. 界面元素

- 评分滑块（1-5 分）
- 数字输入框
- 星级显示区域
- 文字评价输入框
- 提交/取消按钮

(八)、`registerdialog` 函数

用户注册对话框功能说明

1. 功能概述

- 提供用户注册界面
- 包含账号、密码、邮箱等必填信息
- 实现密码一致性验证
- 支持用户名查重功能

2. 界面布局

- 采用表单式设计(`FormLayout`)

- 包含 4 个输入框：

用户名输入框

密码输入框(密文显示)

密码确认框(密文显示)

邮箱输入框

2. 核心功能

- 密码验证：确保两次输入一致

- 用户名查重：防止重复注册

- 数据存储：注册成功保存至全局用户列表

4.交互流程

1) 用户填写注册信息

2) 点击注册按钮

3) 系统验证输入有效性

4) 显示操作结果提示

5) 成功则关闭对话框

三、小组成员分工情况

前端：张景瑜，王泽恺

后端与整合：王泽恺，朱正雍

四、项目总结与反思

（一）、项目概述

目标达成情况：

成功实现跨平台二手交易核心功能（商品管理、社交互动、账户系统）

采用 Qt6.8 技术栈实现预期现代化 UI 设计目标

（二）、成果亮点

技术实现

创新应用 Qt Quick 实现流畅交互动画（商品浏览/收藏特效）

独创的混合渲染方案（QML+Widgets）平衡性能与开发效率

基于 Qt Multimedia 优化的图片压缩上传模块

（三）、问题与挑战

问题领域	具体表现	解决措施	后续改进
跨平台兼容性	Linux 字体渲染异常	增加字体回滚机制	建立 CI 自动化测试矩阵
消息延迟	私信模块偶发 300ms 延迟	引入 SQLite 缓存层	迁移至 WebSocket 方案
新手引导	30%用户需要二次学习发布流程	增加动态指引动画	开发情景式教学模块

（四）、关键反思

架构决策：

正确性：采用 CMake 构建系统大幅提升多平台编译效率

不足：初期低估了 QML 与 C++交互的复杂度，导致 XX 模块重构

团队协作：

高效的 Git 分支策略（Git Flow）保障了 3 人并行开发